

Speech

Hi everyone. I am Hengfeng Wei from Hunan University.

This is joint work with Si Liu and Yuxing Chen.

This work develops a semantic and graph-theoretic foundation for transactional consistency with predicates.

Let me start with the key distinction between item operations and predicate operations.

Look at the employee table.

An item read names a key and returns a value.

The predicate "Salary below 9" instead returns both X and Z, while Y is omitted.

Now, let's review what's well understood without predicates.

For item operations, serializability corresponds to serial execution, while snapshot isolation combines snapshot reads with write-conflict control.

Axiomatic spec frameworks formally specify these models.

Dependency graphs summarize read, write, and anti-dependencies.

We also have the SER characterization theorem.

Roughly, an item-only history is serializable exactly when its dependency graph is acyclic.

For SI, every cycle must contain at least two adjacent anti-dependencies.

These characterizations support history checking, protocol verification, and program robustness analysis.

So without predicates, we have a clean chain from specs, to graphs, to characterizations, and then to downstream analysis.

Now consider predicates.

The intuitive or operational views of SER and SI still apply.

Recent predicate-aware semantics model SQL operations rigorously, but do not establish the dependency-graph correspondence.

In this example, transaction T returns X and Z, while Y is omitted because the visible version has salary 10. Later, S writes salary 9, so Y would now match.

The dependency graph captures this change via the PredRW edge from T to S.

But existing work captures this differently:

some definitions keep all match-changing successors, while others keep only the earliest one or drop the match-changing condition altogether.

So there is no uniform definition of predicate anti-dependency.

Predicate-aware SER graph conditions exist, but lack a rigorous two-way correspondence with predicate-aware semantics.

For SI, Adya's formulation uses lower-level timing information, and its proof relies on item-only properties of SI.

This leaves three missing pieces: a semantic interface, uniform predicate dependencies, and exact SER and SI graph characterizations.

We build that bridge by extending the specification framework, defining predicate-aware dependency graphs, proving SER and SI characterizations, and identifying a design space of predicate anti-dependencies.

The key axiom in the specs is that the latest visible writer must explain either the returned value or the omission.

For example, the visible writer of Y supplies 10, explaining why Y is omitted.

The predicate-read edge records the writer explaining Y .

Since S later changes Y from non-matching to matching, we add a predicate anti-dependency from the reader T to S .

We lift the familiar SER and SI characterizations to histories with predicates.

There are two witness-mismatch issues in the "if" direction proofs: one common to both SER and SI, and one specific to SI.

We omit the details here.

The proofs also show that one precise anti-dependency relation is not mandatory.

We identify an interval in which every admissible relation preserves the same existential SER and SI characterization.

Moreover, the lower bound is sufficient for the "if" direction, while the upper bound is safe for the "only if" direction.

Our theory enables downstream analysis.

For example, we can use the lower and upper bounds to guide predicate-aware history checking.

In summary, we build the bridge through predicate-aware semantics, dependency graphs, exact SER and SI characterizations, and a proof-theoretic design space.

This gives downstream analyses a correctness foundation.